RepoChat

Description:
RepoChat is an AI-powered developer tool that allows users to ask questions about any public GitHub repository. It takes in repository code, generates structured embeddings with metadata, and uses a Retrieval-Augmented Generation (RAG) pipeline to provide accurate explanations of the codebase with source references.

Technologies used:

- Frontend
  - Next.js
  - React
  - TypeScript
  - Tailwind
  - React Markdown (for rendering answers)
  - React Query / Fetch
  - Lucide React icons
- Backend
  - Node.js
  - Express
  - TypeScript
  - OpenAI SDK
    - Embedding model - text-embedding-3-small
    - Chat model - gpt-5-mini
  - Tree-Sitter Parser (CST parsing)
  - Git CLI (for repo cloning)
  - Winston (logging)
- Database
  - MongoDB (Atlas recommended)
  - MongoDB Vector Search
- Security
  - HTTPS
  - Rate limiting (express-rate-limiter)
  - Input validation (zod)
  - OpenAI API key protection

Project Setup:

- Root:
  - npm install npm@latest -g
  - mkdir repo-chat && cd repo-chat
  - git init
  - create .gitignore
  - npm init -y
  - create docker-compose.yml
  - configure eslint + prettier
  - create root scripts
  - create README
- Frontend:
  - cd frontend
  - npx create-next-app@latest
  - Install dependencies
    - tailwind
    - lucide-react
    - react-markdown
    - Axios (maybe)
  - Create:
    - .env
    - .gitignore
    - Dockerfile

        - .dockerignore
    - Configure:
        - Tailwind
        - Next.js environment variables
        - API base URL
- Backend:
    - cd backend
    - npm init -y
    - Install dependencies
        - express
        - cors
        - dotenv
        - mongoose
        - openai
        - babel parser
        - simple-git
        - nodemon
        - eslint
        - prettier
    - Create:
        - .env
        - .gitignore
        - Dockerfile
        - .dockerignore
    - Connect and test MongoDB
- DB:
    - MongoDB Atlas
    - Collections:
        - repositories
        - chunks
        - semantic_cache
    - Indexes:
        - Vector index on embeddings
        - TTL index (24 hours)
- Cache
    - Redis
    - caches
        - Ratelimit
        - Previous questions

Supported Languages:

- Typescript (.ts and .tsx)
- JavaScript (.js and .jsx)
- Python
- Java
- Go
- Ruby
- C++ (CPP)
- C# (C-Sharp)
- HTML
- CSS
- Markdown (Readme)

Project Management:

- GitHub for source control
- AWS for hosting
- GitHub Actions for CI/CD
- Draw.io for architecture diagrams

- Trello for development tracking
- Deployment:
    - Elastic Container Registry - ECR
    - Certificate Manager - ACM
    - Load balancer - ALB
    - Secrets Manager
    - Backend - ECS Fargate
    - Frontend - ECS Fargate
    - Winston logs - CloudWatch
    - Performance monitoring and stack trace - AWS X-Ray
    - Redis - ElastiCache
    - MongoDB - Atlas
- Containerised with Docker
- Logging using Winston

Testing Strategy:

- Backend
    - Jest
    - Supertest
    - Tests:
        - ingestion pipeline
        - vector search
        - RAG pipeline
        - API endpoints
- Frontend
    - React Testing Library
    - Cypress
    - Tests:
        - chat UI
        - repo loading
        - message flow

Repo Ingestion:

- Check repo size and db space left before ingestion (make sure to add a multiplier since ingested data will be larger with the embedding and metadata)
- If in the middle of a job and the repo is not too large, evict the oldest repo to free up space
- Repo size limit 500MB - 1 GB
- File type filtering exclude: (add more after research)
    - .git
    - node_modules
    - images
    - Assets
- 7-day last access delete time

Error Handling:

- When repo is too large?
    - The ingestion request gets rejected
- When cloning fails?
    - Retry a set number of times, then give an error message to the user
- When OpenAI rate limits?
    - Default all chat responses to a default apology/info message

Cleanup

- Scheduled cleanup job
    - Run a cron job every few hours that deletes stale repos based on a 7-day last access

Portfolio Enhancement Tips:

- Add example repositories for demo

- Include performance metrics
- Show token usage improvements from reranking
- Show architecture diagrams
- Show pipeline diagrams
- Add video demo
- Recruiters love:
    - RAG architecture
    - AI engineering
    - Next.js full-stack systems

Documentation (readme):

- System architecture
- RAG pipeline explanation
- MongoDB schema
- Setup guide
- Deployment guide
- API endpoints
- CI/CD explanation

Project Architecture and Data Flow:

Repository Ingestion Pipeline

GitHub Repo
↓
Check the repos collection for an existing entry
↓
If it exists → fetch the latest commit SHA from GitHub API and compare
↓
SHA matches → update lastAccessedAt, serve from existing chunks (break pipeline)
↓
SHA differs → re-ingest, update SHA (continue pipeline)
↓
Clone Repo
↓
Scan Files
↓
AST Chunking (have a chunk deduplication step to check for duplicate code, like boilerplate code (maybe))
↓
Metadata Generation
↓
Embedding Creation
↓
Store in MongoDB

Chunks stored with metadata and embeddings.

Query Pipeline

User Question + repoID
↓
Semantic Cache Check
↓
Update lastAccessedAt on the repo
↓
Query Interpreter (extracts keywords using an LLM)
↓
Vector Search (metadata filtered) (if the vector matches are not confident, based on a pre-defined percentage, say, relevant data could not be found, instead of a hallucinated answer)
↓

Post Retrieval Filtering (score threshold, directory filtering, overlap deduplication, noise filter, per-file cap filter)
↓
Chunk Reranking
↓
Context Compression (if context exceeds a certain limit, this will make an LLM call to basically summarize the code chunk)
↓
LLM Answer Generation
↓
Save to Semantic Cache

Features:

Primary Features

- Repository ingestion
  - Input repoID
  - Clone GitHub repo
  - Parse files
  - Chunk code using AST
- Vector database
  - Store code chunks
  - Store embeddings
  - Metadata filtering
- RAG query system
  - Query repository with natural language
  - Vector search
  - Reranking
  - Context compression
- Chat interface
  - Ask questions about the repo
  - Display answer
  - Show referenced files
- Semantic caching
  - Store previous answers
  - Reduce LLM cost
- Automatic data cleanup
  - Run a cron job every few hours that deletes stale repos based on a 7-day last access

Secondary Features

- Source references
  - Show:
    - file path
    - function name
    - class name
- Repo file tree
  - Display repo structure.
- Question history
  - Show previous questions.
- Rate limiting
  - Prevent abuse

Database and File Storage Structure:

- Repositories Collection

```
{
  repoID: string
  repoURL: string
  createdAt: date
}
```

TTL index: 24 hours

- Chunks Collection

```
{
  content: string
  embedding: [number]

  metadata: {
    repoID: string
    filePath: string
    functionName: string
    className: string
    language: string
    directory: string
}

  createdAt: date
}
```

TTL index: 24 hours

- Semantic Cache

```
{
  repoID: string
  question: string
  answer: string
  createdAt: date
}
```

Communication Rules:

- Handlers can call
  - Controllers
- Controllers can call:
  - Multiple services
  - Format responses
  - Controllers should NOT:
    - Access the database directly
    - Call providers or repositories
- Services can call:
  - Repositories
  - Providers
  - Other services
  - Services should NOT:
    - Know about HTTP or Express
- Repositories can call
  - Database
- Providers can call
  - APIs
    - openAI

Clear Role Definitions:

- Handler
  - Role: API entry point.
  - Responsibilities:
    - Validate request
    - Parse inputs
    - Call controller
    - Return response

- Controller
    - Role: Orchestrate business logic.
    - Responsibilities:
        - Run RAG pipeline
        - Coordinate services
        - Format results
- Service
    - Role: Single-responsibility logic.
    - Examples:
        - cloneRepoService
        - chunkCodeService
        - vectorSearchService
        - rerankService
        - generateAnswerService
- Repositories
    - Role: a repository for each collection that will handle all MongoDB calls for that collection
    - Examples:
        - chunkRepository.ts
        - repoRepository.ts
        - cacheRepository.ts
- Repositories
    - Role: a repository for each api or external service
    - Examples:
        - completionProvider.ts
        - embeddingprovider.ts

File Layout: (including env, git, docker, dockerignore, config folder with prettier and eslint, split into front, back, and DB, tests)

project-root/

.git/
.gitignore
README.md
docker-compose.yml
package.json

backend/
|
|-Dockerfile
|-.dockerignore
|-package.json
|-.env
|-index.ts
|-.prettierrc
|-nodemon.json
|
|-jobs/
| |-cleanupStaleRepos.ts
| |-index.ts
|
|-middleware/
| |-errorHandler.ts
| |-rateLimiter.ts
|
|-routes/
| |-ask.ts
| |- ingest.ts
|

```
|-controllers/
|   |-ragController.ts
|   |- ingestController.ts
|
|-services/
|   |-cloneRepo.ts
|   |-scanFiles.ts
|   |-chunkCode.ts
|   |-generateMetadata.ts
|   |-embedChunks.ts
|   |-storeChunks.ts
|
|   |-queryInterpreter.ts
|   |-vectorSearch.ts
|   |-rerankChunks.ts
|   |-compressContext.ts
|   |-generateAnswer.ts
|   |- semanticCache.ts
|
|-repositories/       (each repository corresponds to a MongoDB collection and holds all MongoDB queries for that collection)
|   |-chunkRepository.ts
|   |-repoRepository.ts
|   |-cacheRepository.ts
|
|-providers/        (Provider for AI interactions)
|   |-embeddingProvider.ts
|   |-completionProvider.ts
|
|-models/
|   |-chunk.ts
|   |- repo.ts
|
|-db/
|   |- mongoClient.ts
|
|-utils/
|   |- logger.ts
|
|- tests/

frontend/
|
|-Dockerfile
|-.dockerignore
|-package.json
|-.env
|
|-app/
|   |-page.tsx
|   |- repo/[repoID]/page.tsx
|
|-components/
|   |-ChatWindow.tsx
|   |-MessageInput.tsx
|   |-MessageList.tsx
|   |-RepoInput.tsx
|   |- SourceReferences.tsx
|
```

```
|-hooks/
|   |- useChat.ts
|
|-lib/
|   |- api.ts
|
|-styles/
|
|- tests/

database/
|- Dockerfile
|- init/
|   |- init-mongo.js
|- scripts/
| |- seed.js
| |- backup.js

docs/

scripts/
```